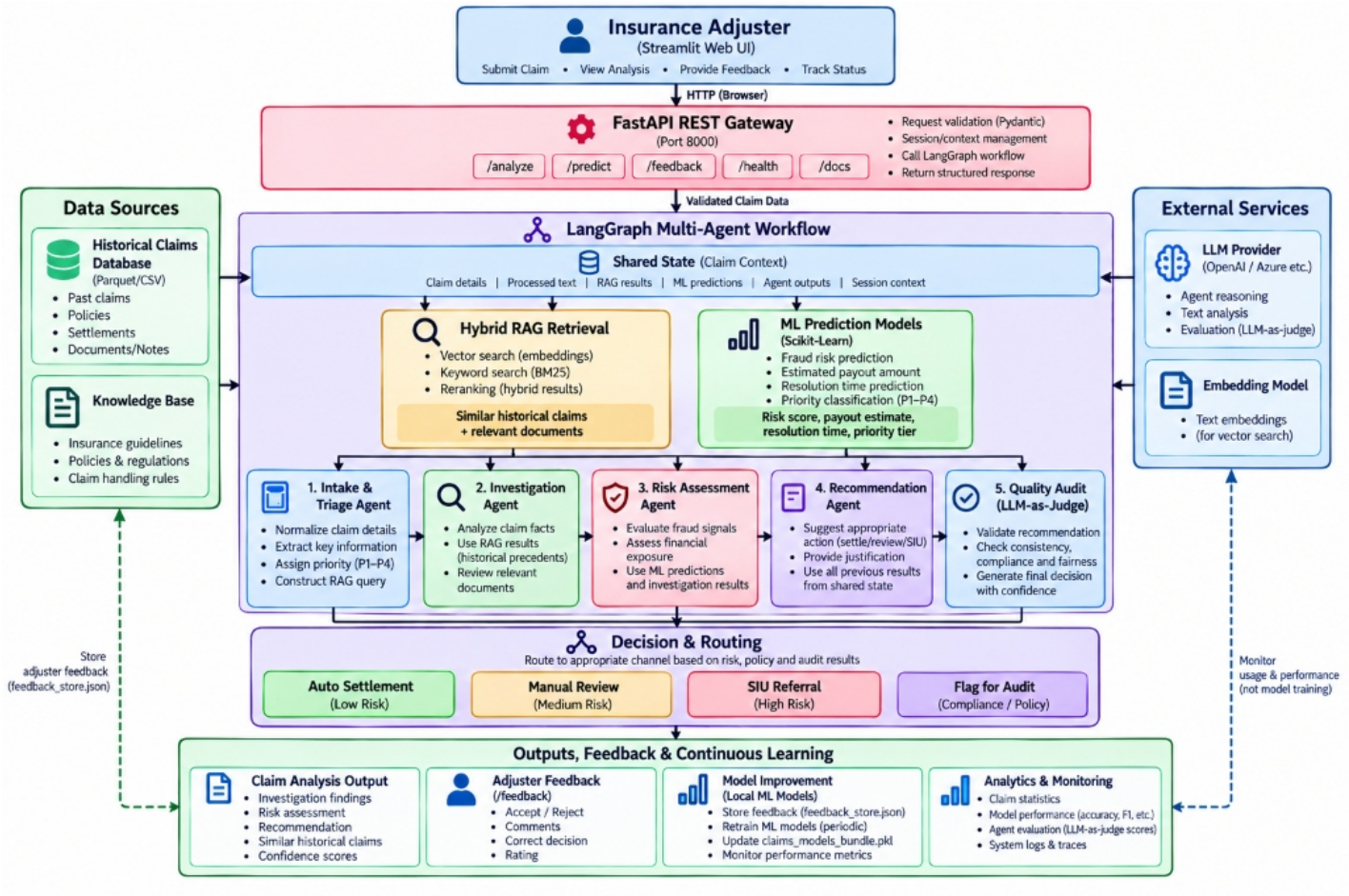
ClaimLens — System Architecture Diagram

End-to-End System Architecture: Client UI → FastAPI Microservice → Guardrails Gate → Multi-Agent Workflow Engine → Hybrid RAG & ML Services → Continuous Feedback Loop



#	Subsystem Component	Architectural Responsibility & Technologies
1	Client & Ingestion Layer	Streamlit Adjuster Dashboard (frontend.py) and FastAPI Gateway (port 8000) exposing all REST routes with /claims/
2	Security & Guardrails	Pydantic schema validation, numeric range checks, date validation, and regex prompt injection sanitization.
3	LangGraph Multi-Agent Workflow	StateGraph with shared ClaimWorkflowState: 1. Intake & Triage -> 2. Hybrid RAG (Vector + BM25) -> 3. Investigatio
4	Dynamic Decision Router	route_after_audit evaluates audit results to route claims to SIU Referral ($0 hold), Supervisor Audit, Manual Adjuster
5	Data & Caching Layer	Historical Claims Dataset (Hugging Face 100M sample), Vector Cache (claim_vectors_cache.pkl), ML Model Bundle
6	Human-in-the-Loop Feedback	Adjuster review interface logging overrides and notes to feedback_store.json.
7	Continuous Retraining Pipeline	Automated holdout evaluation on Golden Benchmark set, Brier score gate, and zero-downtime hot reload updating th
8	Final Standardized Output	Standardized JSON payload returned to client containing classification, precedents, ML estimates, recommendation,

Architectural Invariants & Security: Deterministic financial settlement bounds (payout capped at max(0, claim_amount - deductible)), calibrated fraud risk distribution, and independent LLM-as-Judge audit pass score of 10.0/10.